

DAY 29 ASSIGNMENT

CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

Name:- Rishabh Raj

1. What is Cross-Site Scripting (XSS)?

Cross-Site Scripting (XSS) is a web security vulnerability in which untrusted input is included in a web page in a way that allows unintended client-side script to execute in a user's browser. It can affect confidentiality, integrity, and user sessions.

2. Explain reflected XSS.

Reflected XSS occurs when attacker-controlled input is sent in a request and the application immediately includes that input in its response without appropriate context-aware protection. The payload is not permanently stored by the application.

3. Explain stored XSS.

Stored XSS occurs when malicious or untrusted input is saved by an application, such as in a comment or profile field, and later displayed to other users without proper protection. It can affect every user who views the affected content.

4. Explain DOM-based XSS at a beginner level.

DOM-based XSS happens mainly in the browser when JavaScript reads attacker-controlled data, such as from a URL, and places it into an unsafe DOM location. The vulnerable behavior may occur without the server returning the injected content.

5. What is reflected input?

Reflected input is data supplied by a user that is returned or displayed by the application in the immediate response, for example a search term appearing on a results page.

6. Why does reflected input alone not automatically prove XSS?

Seeing an input value reflected only proves that the application returned the value. XSS requires that the value reaches an executable or unsafe browser context without adequate protection. Proper encoding or sanitization can make reflected input harmless.

7. What is a canary value?

A canary value is a unique, harmless test string used to trace whether supplied input is reflected, stored, or otherwise processed. Example: XSS-CANARY-29.

8. What is output encoding?

Output encoding converts characters into a safe representation appropriate for the context in which they are inserted, such as HTML text or an HTML attribute. It prevents data from being interpreted as markup or code.

9. Why must output encoding match the output context?

Different contexts have different parsing rules. HTML text, HTML attributes, JavaScript, CSS, and URLs require different protections. Encoding for the wrong context may leave special characters meaningful and fail to prevent injection.

10. What is HTML sanitization?

HTML sanitization removes or safely transforms dangerous markup while allowing an approved subset of HTML. It is useful when an application intentionally permits some HTML formatting.

11. What is Content Security Policy (CSP)?

CSP is a browser-enforced security policy delivered mainly through HTTP response headers. It restricts where scripts and other resources can load from and can reduce the impact of some XSS attacks.

12. Why should CSP be treated as Defense in Depth rather than the only XSS defense?

CSP can reduce exploitability, but it depends on correct configuration and browser enforcement and may have gaps or exceptions. The primary defenses should still include safe output encoding, safe DOM APIs, input validation where appropriate, and secure application design.

13. How can HttpOnly reduce some XSS impact?

The HttpOnly cookie attribute prevents JavaScript from directly reading the associated cookie through document.cookie. If XSS occurs, this can reduce the ability to steal session cookies that are protected by HttpOnly.

14. Does HttpOnly remove the XSS vulnerability itself? Explain.

No. HttpOnly protects certain cookies from JavaScript access; it does not stop injected scripts from executing. An XSS attack may still perform actions in the victim's browser using the victim's active session, so the underlying XSS must still be fixed.